

Rule Variant Restrictions for the Tamarin Prover

FELIX LINKER, Department of Computer Science, ETH Zurich, Switzerland

We introduce an optimization to the Tamarin prover that reduces its search space. The optimization applies to protocol models that use equational theories with cancellative operators, for example, when modelling Diffie-Hellman groups or bilinear pairings. We prove the optimization's soundness and evaluate its performance.

1 Introduction

The Tamarin prover [3, 5] is a model checker for security protocol verification, which incorporates a constraint solving algorithm based on symbolic backwards search. It was originally introduced in two doctoral theses [2, 4]. Tamarin operates in the so-called *symbolic model* where cryptographic messages are represented by terms, and cryptographic operations by equations on these terms. The set of equations of a Tamarin theory are the theory's equational theory, and all reasoning happens modulo this equational theory. Tamarin only considers equational theories with specific properties. These properties allow Tamarin to:

- (1) represent all terms in a unique normal form, and
- (2) reason modulo the equational theory by considering finite sets of *variants* of terms.

These variants of terms are computed in a pre-computation phase. In this paper, we introduce a pruning mechanism for variants that significantly reduces Tamarin's search space and prove this mechanism's soundness. We experienced significant performance improvements when using this pruning mechanism, in particular when modelling protocols that use Diffie-Hellman group operators.

The purpose of this paper is to document this optimization, in particular, its soundness proof for the maintenance of the Tamarin prover. As of publishing, our optimizations have already been integrated into Tamarin's code base.¹

Structure. We proceed as follows. We introduce relevant background in Section 2. Then, in Section 3, we introduce our optimization and prove its soundness by relating it to a soundness proof in one of the original theses describing Tamarin. We evaluate our optimization's performance in Section 4.

2 Background

In this paper, we introduce the Tamarin prover following [2]. We omit certain details for clarity. For full details, we refer the reader to [2].

2.1 Preliminaries

An *order-sorted signature* $\Sigma = (S, \leq, F)$ consists of a set of sorts S , a partial order $(\leq) \subseteq S \times S$ on the sorts, and a set of function symbols F . Every sort $s \in S$ has an infinite set of associated variables $\mathcal{V}_s$ and constants C_s , which are all pairwise disjoint. We write $v:s$ for $v \in \mathcal{V}_s$ and $\mathcal{V} = \bigcup_{s \in S} \mathcal{V}_s$ for the set of all variables. Functions in F have the form $f : s_1 \times \dots \times s_n \rightarrow s$ (where $s_1, \dots, s_n, s \in S$). We require that every connected component C in $(S, \leq)$ has a supremum, which we denote by $\text{top}(s)$ for $s \in C$. Furthermore, we require that if $f : s_1 \times \dots \times s_n \rightarrow s \in F$ then $f : \text{top}(s_1) \times \dots \times \text{top}(s_n) \rightarrow \text{top}(s) \in F$. For $s \in S$, we use $\mathcal{T}_\Sigma^s$ to denote the terms of sort s . The set of all terms of an order-sorted signature is $\mathcal{T}_\Sigma = \bigcup_{s \in S} \mathcal{T}_\Sigma^s$. A term is *ground* if it contains no variables.

¹See <https://github.com/tamarin-prover/tamarin-prover/pull/763>.

A substitution is a function $\sigma : \mathcal{V} \rightarrow \mathcal{T}_\Sigma$ mapping variables to terms. We require that substitutions respect sorts, i.e., variables of sort s must be mapped to terms of sort $s' \leq s$. We extend substitutions homomorphically to terms and other constructs (to be defined later) as usual, and write $t\sigma$ for an application of σ to a term $t \in \mathcal{T}_\Sigma$. A *valuation* is a substitution to ground terms. Valuations can also be defined as explicit lists. In that case, variables not mapped in that list are irrelevant and can map to arbitrary terms.

2.2 Protocol Specifications

Tamarin works in the *symbolic model*, where protocol messages are represented as elements of a term algebra. An equational theory encodes the semantics of these terms. Along with the equational theory, protocol and attacker behavior are specified by a set of multiset rewriting rules. A Tamarin protocol model is a pair of an equational theory and a set of multiset rewriting rules.

2.2.1 Equational Theories. Protocol models are defined with respect to an order-sorted signature Σ and a set of equations E that have the form $s \simeq t$. Typically, the equations model cryptographic operations and their relationships. $\Xi = (\Sigma, E)$ is an *equational presentation*. An *equational theory* $=_\Xi$ is the smallest congruence relation over Σ containing all instances of E . We often leave Σ implicit and identify Ξ by E . Two terms s and t are *equal modulo E* if $t =_E s$. We will use the E -subscript to denote operators, e.g., $\in_E$ modulo E .

In the following, we fix the sorts in Σ to *msg*, *fresh*, *pub*, and *nat*. *fresh*, *pub*, and *nat* are subsorts of *msg*. The sort *fresh* models fresh, unguessable values such as cryptographic random numbers. We sometimes prefix variables with a sort-specific symbol. Variables in *fresh* are prefixed with $\sim$, variables in *pub* with $\$$, and variables in *nat* with $\%$. When not explicitly mentioned or clear from context, a variable without prefix or sort annotation is of sort *msg*.

2.2.2 Term Rewriting. A *position* p in a term is a list of indices. We write $t|_p$ for t 's *subterm at position p* , and define it inductively as:

$$t|_p := \begin{cases} t & p = [] \\ t_{i_0}|_{[i_1, \dots, i_n]} & p = [i_0, i_1, \dots, i_n], t = f(t_1, \dots, t_k), 1 \leq i_0 \leq k \\ \perp & \text{otherwise} \end{cases}$$

We write $t[s]_p$ to denote the term obtained from replacing t_p in t with s . We say that t is a *syntactic subterm* of t' if there exists a position p such that $t'|_p = t$. Whenever this is the case, we write $t \sqsubseteq t'$.

A *rewrite rule* is an ordered pair of terms $l \rightarrow r$. A *rewriting system* $\mathcal{R}$ is a set of rewrite rules. The corresponding *rewrite relation* $\rightarrow_{\mathcal{R}}$ is defined as $s \rightarrow_{\mathcal{R}} t$ if there is a position p in s , a rewrite rule $l \rightarrow r \in \mathcal{R}$, and a substitution σ such that $s|_p = l\sigma$ and $t = s[r\sigma]_p$.

A rewriting system $\mathcal{R}$ is *terminating* if there is no infinite sequence of terms $(t_i)_{i \in \mathbb{N}}$ such that $t_i \rightarrow_{\mathcal{R}} t_{i+1}$ ($i \in \mathbb{N}$). $\mathcal{R}$ is *confluent* when for all terms t , s_1 , and s_2 , if $t \rightarrow_{\mathcal{R}}^* s_1$ and $t \rightarrow_{\mathcal{R}}^* s_2$ then there exists a term t' such that $s_1 \rightarrow_{\mathcal{R}}^* t'$ and $s_2 \rightarrow_{\mathcal{R}}^* t'$. $\mathcal{R}$ is *convergent* if it is terminating and confluent. We use $t \downarrow_{\mathcal{R}}$ to denote a term t 's normal form in a convergent rewriting system $\mathcal{R}$, which is the term t' such that $t \rightarrow_{\mathcal{R}}^* t'$ and there exists no term t'' such that $t' \rightarrow_{\mathcal{R}} t''$. Observe that every term's normal form is uniquely determined.

If an equational theory E 's equations can be oriented to a convergent rewriting system $\mathcal{R}$, then $t =_E s$ if and only if $t \downarrow_{\mathcal{R}} = s \downarrow_{\mathcal{R}}$. Some equations, e.g., commutativity, cannot be oriented and in those cases, we split an equational theory E into oriented rewriting rules $\mathcal{R}$ and unoriented equations AX . The *rewrite relation* $\rightarrow_{\mathcal{R}, AX}$ is defined as $s \rightarrow_{\mathcal{R}, AX} t$ if there is a position p , a rewrite rule $l \rightarrow r \in \mathcal{R}$, and a substitution σ such that $s|_p =_{AX} l\sigma$ and $t = s[r\sigma]_p$. We say that

$\mathcal{R}$ is AX -convergent if $\rightarrow_{\mathcal{R},AX}$ is terminating and confluent. We write $t \downarrow_{AX}^{\mathcal{R}}$ to denote t 's normal form with respect to $\rightarrow_{\mathcal{R},AX}$. $\mathcal{R}$ is AX -coherent when for all terms t_1, t_2 , and t_3 , if $t_1 \rightarrow_{\mathcal{R},AX}^* t_2$ and $t_1 =_{AX} t_3$ then there exist terms t_4, t_5 such that $t_4 =_{AX} t_5$ and $t_2 \rightarrow_{\mathcal{R},AX}^* t_4$ and $t_3 \rightarrow_{\mathcal{R},AX}^+ t_5$. We write $\mathcal{R}^\approx := \{l \approx r \mid l \rightarrow r \in \mathcal{R}\}$ for a rewriting system $\mathcal{R}$'s associated equational theory. For $E = \mathcal{R}^\approx \cup AX$, if $\mathcal{R}$ is AX -convergent and AX -coherent, then $t =_E s$ if and only if $t \downarrow_{AX}^{\mathcal{R}} =_{AX} s \downarrow_{AX}^{\mathcal{R}}$.

2.2.3 Multiset Rewriting Rules. In Tamarin, state transitions are modeled using multiset rewriting rules. The global state consists of a multiset of ground facts, and the rewriting rules define how the state is updated. Facts are similar to predicates in first-order logic and are of the form $F(t_1, \dots, t_n)$. F is a fact symbol from a fact signature Σ_{Fact} , and $t_1, \dots, t_n$ are terms defined over the model's signature Σ . A fact is ground if all its terms are ground. Facts typically model the local state of participants or the network, e.g., who possesses which keys, or what messages have been sent over the network. We consider the special, reserved fact symbols Fr , In , Out , $!K^\uparrow$, and $!K^\downarrow$ of arity 1 in Σ_{Fact} . These symbols respectively model generating a fresh, unguessable value, receiving and sending a message over an insecure network, and adversary reasoning.

Multiset rewriting rules in Tamarin have the form

$$[l_1, \dots, l_m] \multimap [a_1, \dots, a_n] \rightarrow [r_1, \dots, r_o].$$

l_i, a_j, r_k are facts and may use free variables ($1 \leq i \leq m, 1 \leq j \leq n, 1 \leq k \leq o$). $l_1, \dots, l_m$ are the rule's *premises*, $a_1, \dots, a_n$ the rule's *actions* (also called *action facts*), and $r_1, \dots, r_o$ the rule's *conclusions*. $[a_1, \dots, a_n]$ can be omitted when a rule has no actions. Rules can be instantiated by a substitution σ . One rule ri is another rule r 's *instance* if there exists a substitution σ such that $r\sigma = ri$. We write $prems(r)$, $acts(r)$, and $concs(r)$ for a rule r 's list of premises, actions, and conclusions respectively. A rule (instance) is ground if all its facts are ground.

Facts are either persistent (preceded by a $!$) or linear. When a rule is applied, the linear facts in its premise are removed from the global state and replaced by the facts in its conclusion. Persistent facts are not consumed when used in a rule's premise.

A set of multiset rewriting rules P is a *multiset rewriting system* if

- (i) no rule uses a fresh constant,
- (ii) no rule's premise contains an Out , $!K^\uparrow$, or $!K^\downarrow$ fact, and
- (iii) no rule's conclusion contains a Fr , In , $!K^\uparrow$, or $!K^\downarrow$ fact, and
- (iv) all *fresh*, *msg*, and *nat* variables used in a rule's conclusion are introduced in a rule's premise.

Built-In Rules. Tamarin uses a number of built-in rules to model the generation of random values and the adversary deduction following the model's equational theory E , the latter of which we refer to by ND (normal form message deduction rules). The following rules model generating fresh values:

$$FRESH := [] \rightarrow [Fr(x:fresh)].$$

Notably, the following rule is in ND :

$$[Fr(x:fresh)] \multimap [!K^\uparrow(x:fresh)] \rightarrow [!K^\uparrow(x:fresh)].$$

2.3 Semantics

2.3.1 Executions and Traces. Given a multiset rewriting system P , adversary deduction rules ND , an equational theory E , and $R = P \cup ND \cup \{\text{FRESH}\}$, the tuple (R, E) is a *Tamarin* or *protocol model*. Tamarin models induce a labeled state transition system. The initial state is the empty multiset $\emptyset^\#$. A state transition is the application of a ground rule instance $ri = l \multimap a \mapsto r$. We use $\text{lfacts}(x)$ and $\text{pfacts}(x)$ to denote the linear and persistent facts respectively in x . ri can be applied if its premises are contained in the global state, which is then updated accordingly. Formally, the state transition relation $\Rightarrow_{(R,E)}$ is defined as:

$$S \Rightarrow_{(R,E)} (S \setminus \text{lfacts}(l)) \cup^\# r \quad \text{if } \text{lfacts}(l) \subseteq^\# S, \text{pfacts}(r) \subseteq \text{set}(S)$$

An *execution* is a sequence of labeled state transitions. We only consider executions where each instance of **FRESH** is unique. Each execution has a corresponding *trace*, which is the list of sets of action facts labeling each transition. The semantics of a Tamarin model is its set of traces.

2.3.2 Dependency Graphs. Dependency graphs capture the sequence of rule applications and dependencies between rule premises and conclusions. They are closely related to the constraint systems used in Tamarin's protocol analysis, and they enable effective proof search.

DEFINITION 1 (DEPENDENCY GRAPH). Let (R, E) be a Tamarin model. A dependency graph is a tuple $dg = (I, D)$ where I is a finite list of ground rule instances in R , and $D \subseteq \mathbb{N}^2 \times \mathbb{N}^2$. A rule's index in I is its timepoint. We write $(i, u) \mapsto (j, v)$ for $((i, u), (j, v)) \in D$ when D is clear from the context. Here, (i, u) denotes the conclusion with index u of the rule with index i in I . Similarly, (j, v) denotes the premise with index v of the rule with index j . We require that edges correctly connect premises and conclusions, i.e., $i < j$ and the conclusion (i, u) is equal modulo E to the premise (j, v) . Furthermore, each premise must have exactly one incoming edge, every linear conclusion has at most one outgoing edge, and instances of **FRESH** are unique.

DEFINITION 2 (TRACE). A trace induced by a dependency graph $dg = (I, D)$ is a list $\text{trace}(dg)$ of sets of action labels. Concretely, $\text{trace}(dg)$ is of length $|I|$ and every index i corresponds to the set of action labels at timepoint i , i.e., to $\text{set}(\text{acts}(I_i))$.

One can show that the dependency graphs of a model (R, E) induce the same set of traces as the set of traces derived from the model's executions ([2, Theorem 3]). We write $\text{dgraphs}_E(R)$ for all dependency graphs of a protocol model (R, E) and $\text{trace}(dg)$ for a dependency graph's associated trace.

2.4 Protocol Properties

In Tamarin, protocol properties are expressed as first-order logic *trace properties*, which are given by closed *trace formulas*. To reason about temporal ordering of events, we introduce a new sort *tmp*, incomparable to *msg*, for temporal values and variables. The constants of *tmp* are the elements of $\mathbb{Q}$.

For a fact f , temporal variables i and j representing timepoints, and terms t and u , atomic trace formulas are:

- (1) false $\perp$,
- (2) action formulas $f@i$, and
- (3) the predicates for
 - (a) timepoint equality $i \doteq j$,
 - (b) timepoint ordering $i < j$,
 - (c) term equality $t = u$, and

(d) subterm relation $t \sqsubset u$ (also written $t \ll u$; introduced in [1]).

Trace formulas can be combined with the logical connectives $\neg$ and $\wedge$, and *fresh* and *msg* variables can be existentially quantified. As usual, we use $\vee$, $\implies$ and universal quantification as derived connectives. We write $fv(\varphi)$ for the free variables of a formula φ . A trace formula is *guarded* if all occurrences of existential or universal quantifiers are of the form $\exists \vec{x}. f@i \wedge \varphi$ or $\forall \vec{x}. f@i \implies \varphi$, where $\text{set}(\vec{x}) \subseteq fv(f@i)$, i.e., all bound variables are free in the action formula $f@i$.

Given a valuation θ of the free variables of φ , we write $(tr, \theta) \models_E \varphi$ to mean that the trace tr satisfies φ under the valuation θ . Note that we interpret variables of sort s in the set of ground terms of sort s . As all functions in Σ (except for $+$) are of sort *msg*, this means that variables of sort *msg* and *nat* are interpreted as ground terms, while *fresh*, *tmp*, and *pub* are interpreted as constants. We do not formally define $\models_E$ but refer to [2].

A trace formula φ is *valid* for a protocol model (R, E) , written $R \models_E^\forall \varphi$, if for all traces tr of (R, E) and every valuation θ , one has $(tr, \theta) \models_E \varphi$. φ is *satisfiable* in (R, E) , written $R \models_E^\exists \varphi$, if there exists a trace tr of (R, E) and valuation θ such that $(tr, \theta) \models_E \varphi$.

2.5 Normal Form Dependency Graphs

Let E be an equational theory, $\mathcal{R}$ be a rewriting system, and $AX \subseteq E$ the equations for associativity and commutativity in E such that (1) $E = \mathcal{R}^\approx \cup AX$ and (2) $\mathcal{R}$ is AX -convergent and AX -coherent. E has the *finite variant property* if for all terms t there is a finite set of substitutions $T = \{\tau_1, \dots, \tau_k\}$ such that for all substitutions σ there is a substitution $\tau \in T$ and substitution σ' so that $t\sigma \downarrow_{AX}^\mathcal{R} =_{AX} (t\tau \downarrow_{AX}^\mathcal{R})\sigma'$ and for all variables x occurring in t , we have $x\sigma \downarrow_{AX}^\mathcal{R} =_{AX} x\tau\sigma'$. The set $\lceil t \rceil_{AX}^\mathcal{R} := \{(t\tau_i \downarrow_{AX}^\mathcal{R}, \tau_i) \mid 1 \leq i \leq k\}$ is a *complete set of $\mathcal{R}$, AX -variants of t* . We extend the notion of complete variants to multiset rewriting rules and multiset rewriting systems straightforwardly. Intuitively, the variants of a term or rule allow us to reason modulo E by reasoning modulo normal forms and AX only.

THEOREM 2.1 (CHANGING THE EQUATIONAL THEORY [2, P.114]). *For every guarded trace property φ and protocol model (R, E) where $\mathcal{R}^\approx \cup AX = E$:*

$$R \models_E^\forall \varphi \iff \{ \text{trace}(dg) \mid dg \in \text{dgraphs}_{AX}(\lceil R \rceil_{AX}^\mathcal{R}), dg \text{ is } \downarrow_{AX}^\mathcal{R}\text{-normal} \} \models_{AX}^\forall \varphi.$$

3 Rule Variant Restrictions

Our optimization applies to theories that use operators which can cancel out and thus have multiple variants attached to them. Consider, for example, the following protocol rule using Diffie-Hellman exponentiation:

$$[\text{Fr}(x:\text{fresh}), \text{In}(m)] \rightarrow [\text{Out}(m^x)].$$

During precomputation, Tamarin will consider that exponentiation might cancel out, leading to the rule variant ($\circ^{-1}$ denotes an element's inverse):

$$[\text{Fr}(x:\text{fresh}), \text{In}(z^{x:\text{fresh}^{-1}})] \rightarrow [\text{Out}(z)].$$

It makes sense that Tamarin considers this variant during precomputation as it considers this rule in isolation. However, when one additionally considers the semantics of Tamarin protocol models, one can see that this rule variant could never occur in a normal form dependency graph. This is because the value that is used in the exponentiation, $x:\text{fresh}$, is freshly introduced in this rule, but also required in the term m received over the insecure network. To construct this term, the sender must have had access to $x:\text{fresh}$ before it was introduced, which is impossible.

In this section, we formalize the above intuition, and introduce *fresh-contradictory variants*, that is, variants which need to use a fresh value before it was introduced. We show that one need not consider such variants during proof search as the set of traces (normal form dependency graphs) admitted by a protocol model does not change when removing fresh-contradictory rule variants.

DEFINITION 3. Let $r = [p_1, \dots, p_n] \dashv [\dots] \rightarrow [\dots]$ be a protocol rule and $\lceil r \rceil_{AX}^{\mathcal{R}} = \{(r_i, \tau_i)\}_{i \in \mathbb{N}}$ a set of associated variants for a rewriting system $\mathcal{R}$ and associate and commutative equations AX . A variant $(r_i, \tau_i) \in \lceil r \rceil_{AX}^{\mathcal{R}}$ is *fresh-contradictory* if r has two distinct premises $p_i = F(\dots, t, \dots)$ ($F \neq \text{Fr}$) and $p_j = \text{Fr}(x)$ such that $x\tau_i \sqsubseteq t\tau_i \downarrow_{AX}^{\mathcal{R}}$.

We write $\lceil t \rceil_{AX}^{\mathcal{R}, \perp} \subseteq \lceil t \rceil_{AX}^{\mathcal{R}}$ for the fresh-contradictory variants of a term t . Again, we extend this notion to multiset rewriting rules and sets of multiset rewriting rules.

LEMMA 3.1. For every protocol model (R, E) where $\mathcal{R}^{\approx} \cup AX$ is an equational representation for $=_E$ and AX only contains equations for associativity and commutativity:

$$\{dg \downarrow_{AX}^{\mathcal{R}} \mid dg \in dgraphs_{AX}(\lceil R \rceil_{AX}^{\mathcal{R}})\} = \{dg \downarrow_{AX}^{\mathcal{R}} \mid dg \in dgraphs_{AX}(\lceil R \rceil_{AX}^{\mathcal{R}} \setminus \lceil R \rceil_{AX}^{\mathcal{R}, \perp})\}.$$

PROOF. Let

$$\begin{aligned} NDG &:= \{dg \downarrow_{AX}^{\mathcal{R}} \mid dg \in dgraphs_{AX}(\lceil R \rceil_{AX}^{\mathcal{R}})\} \\ NDG_{-\perp} &:= \{dg \downarrow_{AX}^{\mathcal{R}} \mid dg \in dgraphs_{AX}(\lceil R \rceil_{AX}^{\mathcal{R}} \setminus \lceil R \rceil_{AX}^{\mathcal{R}, \perp})\}. \end{aligned}$$

It is clear that $NDG_{-\perp} \subseteq NDG$. We show that $NDG \subseteq NDG_{-\perp}$ by contradiction. Presume that there is a dependency graph $dg = (I, D) \in NDG \setminus NDG_{-\perp}$. By assumption, we know that all rule instances $ri \in I$ are $\downarrow_{AX}^{\mathcal{R}}$ -normal (as dg is $\downarrow_{AX}^{\mathcal{R}}$ -normal). Thus, by the finite variant property, we know that for each rule instance $ri \in I$ there must be a rule variant $(r, \tau) \in \lceil R \rceil_{AX}^{\mathcal{R}}$ so that $ri =_{AX} r\sigma$ for some substitution σ (note that r is $\downarrow_{AX}^{\mathcal{R}}$ -normal by definition). There must be some rule instance ri for which its associated variant (r, τ) (as per above) is in $\lceil R \rceil_{AX}^{\mathcal{R}, \perp}$. If not, dg would be in $NDG_{-\perp}$. We fix such a rule instance ri and associated variant (r, τ) .

As r is a fresh-contradictory variant, there must be some fresh variable x and term t occurring in $r\sigma$ such that $x \sqsubseteq t\sigma$ and x is introduced in r , i.e., $r\sigma$ has a $\text{Fr}(x)$ premise. Observe that $x \sqsubseteq ri$ since $ri =_{AX} r\sigma$ and equations in AX can never remove variables from a term. As ri is $\downarrow_{AX}^{\mathcal{R}}$ -normal (cannot be reduced), t in ri must have been constructed using x . Therefore, there must be another rule instance that introduces x as a fresh variable to construct t and thus also contains a premise $\text{Fr}(x)$. This, however, contradicts that instances of FRESH are unique. $\square$

We can use this lemma to specialize Theorem 2.1 into the following one.

THEOREM 3.2. For every guarded trace property φ and protocol model (R, E) where $\mathcal{R}^{\approx} \cup AX$ is an equational representation for $=_E$ and AX only contains equations for associativity and commutativity:

$$R \models_E^{\forall} \varphi \iff \{trace(dg) \mid dg \in dgraphs_{AX}(\lceil R \rceil_{AX}^{\mathcal{R}} \setminus \lceil R \rceil_{AX}^{\mathcal{R}, \perp}), dg \text{ is } \downarrow_{AX}^{\mathcal{R}}\text{-normal}\} \models_{AX}^{\forall} \varphi.$$

PROOF. Follows directly from Theorem 2.1 and Lemma 3.1. $\square$

Theorem 3.2 allows us to prune all fresh-contradictory variants during Tamarin's precomputations.

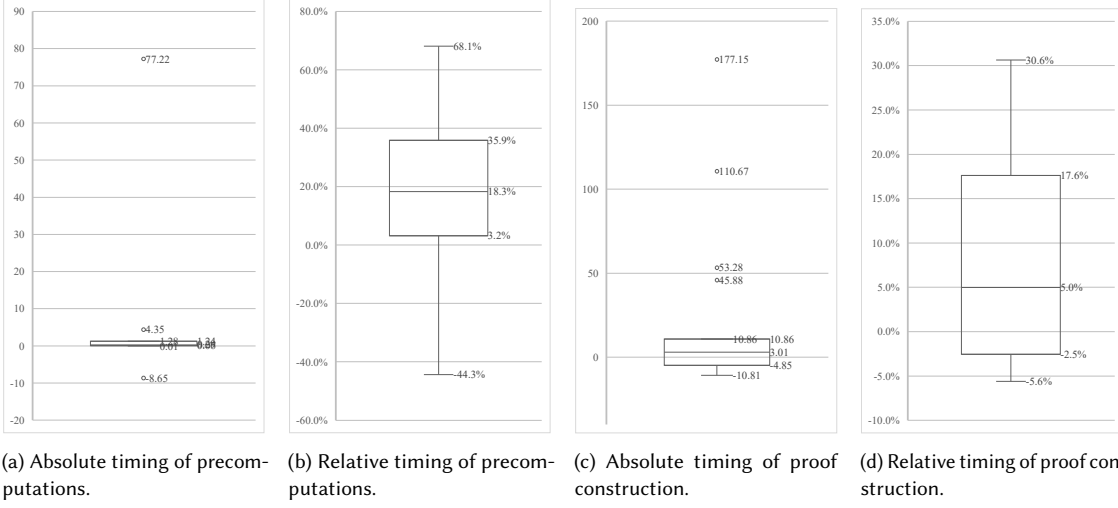


Fig. 1. Benchmarks of the optimization. Each diagram displays the absolute or relative time difference when enabling the optimization. Higher is better.

4 Evaluation

We evaluated the performance change of our optimization on 19 authenticated key exchange examples that use Diffie-Hellman and bilinear pairings. These examples were taken from the Tamarin prover repository [6]. We timed both precomputations and proof construction (which includes precomputations) with and without our optimization, taking the average over 6 runs on a laptop with an Apple M2 Max CPU and 32 GB of RAM.

Figure 1 shows the benchmark results as a box-and-whisker diagram. Generally, the optimization does improve performance, though not always. As for the precomputations, the negative outlier is a theory that does not benefit from the optimization, but that also has many rule variants due to its equational theory. We conjecture that checking for fresh-contradictory rule variants takes extra time but does not benefit the more complex precomputations that follow. As for proof times, we observed that Tamarin’s heuristics prioritize equations with many and few cases differently. For some theories, we encountered performance degradation because Tamarin started solving equations with multiple solutions, although this did not aid the overall proof.

References

- [1] Cas Cremers, Charlie Jacomme, and Philip Lukert. “Subterm-Based Proof Techniques for Improving the Automation and Scope of Security Protocol Analysis”. In: *36th Computer Security Foundations Symposium (CSF)*. July 2023. doi: 10.1109/CSF57540.2023.00001.
- [2] Simon Meier. “Advancing Automated Security Protocol Verification”. Doctoral Thesis. ETH Zurich, 2013. doi: 10.3929/ethz-a-009790675.
- [3] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. “The TAMARIN Prover for the Symbolic Analysis of Security Protocols”. In: *Computer Aided Verification (CAV)*. Berlin, Heidelberg, 2013. doi: 10.1007/978-3-642-39799-8_48.

- [4] Benedikt Schmidt. “Formal Analysis of Key Exchange Protocols and Physical Protocols”. Doctoral Thesis. ETH Zurich, 2012. DOI: 10.3929/ethz-a-009898924.
- [5] Benedikt Schmidt, Simon Meier, Cas Cremers, and David Basin. “Automated Analysis of Diffie-Hellman Protocols and Advanced Security Properties”. In: *25th Computer Security Foundations Symposium (CSF)*. June 2012. DOI: 10.1109/CSF.2012.25.
- [6] *The Tamarin Prover Repository*. Version 4b6239f1386912a2ae15d71c75bcad1bdcafdce1. URL: <https://github.com/tamarin-prover/tamarin-prover/> (visited on 02/11/2026).